МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №4.2
по дисциплине
«Программирование»
по теме: "Динамические структуры данных"

Выполнил студент гр. ИВТб-1301-06-00 _____/Ичетовкин Н.А./
Проверил доцент кафедры ЭВМ _____/Долженкова М.Л./

Киров
2026

Цель

Научиться создавать простые графические интерфейсы с использованием библиотеки tkinter, организовывать обработку событий и интегрировать ранее разработанные программные модули в единое приложение. Закрепить понимание работы выбранной структуры данных через её визуальное представление.

Задание

- На основе реализации структуры данных из лабораторной работы 4.1. необходимо разработать оконное приложение с графическим интерфейсом на Python.
- Все операции со структурой данных должны выполняться исключительно через вызовы функций отдельно оформленных библиотечных модулей.
- Необходимо предусмотреть реализацию, как минимум двух модулей: один на C++, второй — Python.
- Для получения дополнительных баллов, необходима реализация двух вариантов C++ модуля: с использованием структур на динамической памяти и с использованием контейнеров и адаптеров STL.
- Интерфейс должен адекватно реагировать на попытку выполнить некорректную операцию (например, Pop из пустого стека), выводя понятное сообщение, без аварийного завершения программы.
- Визуализации должна обновляться после каждой успешной операции.

Исходный код файла приложения

```
import tkinter as tk
from tkinter import ttk, messagebox
import math
from heap_python_module import PythonHeap
from heap_cpp_wrapper import CppHeapWrapper

class HeapVisualizer:
    """Основной класс визуализации кучи"""

    def __init__(self, root):
        self.root = root
        self.root.title("Визуализатор структуры данных: Куча (Heap)")
        self.root.geometry("900x650")

        # Выбор реализации
        self.heap = PythonHeap()  # По умолчанию Python модуль
        self.current_module = "Python"

        # Цветовая схема
        self.colors = {
            'bg': '#2b2b2b',
            'node': '#4a90d9',
            'node_hover': '#357abd',
            'text': '#ffffff',
            'line': '#888888',
            'button_bg': '#3c3f41',
            'button_fg': '#ffffff',
            'input_bg': '#3c3f41',
            'input_fg': '#ffffff',
            'label_bg': '#3c3f41',
            'label_fg': '#ffffff',
            'error': '#cc5555',
            'success': '#55cc55',
            'menu_bg': '#2d2d2d',
```

```
        'frame_bg': '#323232'
    }

    self.setup_ui()
    self.draw_heap()

def setup_ui(self):
    """Настройка пользовательского интерфейса"""
    self.root.configure(bg=self.colors['bg'])

    # Основной контейнер
    main_container = tk.Frame(self.root, bg=self.colors['bg'])
    main_container.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    # Левая панель с управлением
    left_panel = tk.Frame(main_container, width=250, bg=self.colors['frame_bg'],
                          relief=tk.RAISED, bd=1)
    left_panel.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 10))
    left_panel.pack_propagate(False)

    # Правая панель с визуализацией
    right_panel = tk.Frame(main_container, bg=self.colors['bg'])
    right_panel.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

    # Заголовок левой панели
    header_frame = tk.Frame(left_panel, bg=self.colors['menu_bg'])
    header_frame.pack(fill=tk.X, pady=(0, 10))

    label = tk.Label(header_frame, text="Управление кучей",
                     font=('Arial', 14, 'bold'),
                     bg=self.colors['menu_bg'], fg=self.colors['text'])
    label.pack(pady=10)

    # Выбор модуля
    module_frame = tk.Frame(left_panel, bg=self.colors['frame_bg'])
    module_frame.pack(fill=tk.X, pady=(0, 10))
```

```
tk.Label(module_frame, text="Модуль:",
        bg=self.colors['frame_bg'], fg=self.colors['label_fg']).pack(side=tk.L

self.module_var = tk.StringVar(value="Python")
module_combo = ttk.Combobox(module_frame, textvariable=self.module_var,
                            values=["Python", "C++"], state="readonly", width=
module_combo.pack(side=tk.RIGHT, padx=5, pady=5)
module_combo.bind('<<ComboboxSelected>>', self.on_module_change)

# Операции с кучей
operations_frame = tk.Frame(left_panel, bg=self.colors['frame_bg'])
operations_frame.pack(fill=tk.X, pady=10)

tk.Label(operations_frame, text="Операции:",
        font=('Arial', 11, 'bold'),
        bg=self.colors['frame_bg'], fg=self.colors['label_fg']).pack(pady=5)

# Кнопка добавления элемента
add_frame = tk.Frame(operations_frame, bg=self.colors['frame_bg'])
add_frame.pack(fill=tk.X, pady=5)

tk.Label(add_frame, text="Значение:",
        bg=self.colors['frame_bg'], fg=self.colors['label_fg']).pack(side=tk.L

self.entry_value = tk.Entry(add_frame, bg=self.colors['input_bg'],
                            fg=self.colors['input_fg'], width=10)
self.entry_value.pack(side=tk.RIGHT, padx=5)

button_add = tk.Button(operations_frame, text="Добавить элемент",
                      command=self.push_element,
                      bg=self.colors['button_bg'],
                      fg=self.colors['button_fg'],
                      relief=tk.FLAT, pady=5)
button_add.pack(fill=tk.X, padx=10, pady=5)
```

```
# Кнопка удаления корня
button_pop = tk.Button(operations_frame, text="Удалить корень",
                       command=self.pop_element,
                       bg=self.colors['button_bg'],
                       fg=self.colors['button_fg'],
                       relief=tk.FLAT, pady=5)
button_pop.pack(fill=tk.X, padx=10, pady=5)

# Удаление по индексу
remove_frame = tk.Frame(operations_frame, bg=self.colors['frame_bg'])
remove_frame.pack(fill=tk.X, pady=5)

tk.Label(remove_frame, text="Индекс:",
        bg=self.colors['frame_bg'], fg=self.colors['label_fg']).pack(side=tk.L

self.entry_index = tk.Entry(remove_frame, bg=self.colors['input_bg'],
                            fg=self.colors['input_fg'], width=10)
self.entry_index.pack(side=tk.RIGHT, padx=5)

button_remove = tk.Button(operations_frame, text="Удалить по индексу",
                          command=self.remove_at_index,
                          bg=self.colors['button_bg'],
                          fg=self.colors['button_fg'],
                          relief=tk.FLAT, pady=5)
button_remove.pack(fill=tk.X, padx=10, pady=5)

# Разделитель
separator = ttk.Separator(operations_frame, orient='horizontal')
separator.pack(fill=tk.X, padx=10, pady=10)

# Кнопка очистки
button_clear = tk.Button(operations_frame, text="Очистить кучу",
                         command=self.clear_heap,
                         bg='#cc5555',
                         fg=self.colors['text'],
                         relief=tk.FLAT, pady=5)
```

```
        button_clear.pack(fill=tk.X, padx=10, pady=5)

        # Информация о куче
        info_frame = tk.Frame(left_panel, bg=self.colors['frame_bg'])
        info_frame.pack(fill=tk.X, pady=10)

        tk.Label(info_frame, text="Информация:",
                font=('Arial', 11, 'bold'),
                bg=self.colors['frame_bg'], fg=self.colors['label_fg']).pack(pady=5)

        self.info_text = tk.Text(info_frame, height=6, width=30,
                                 bg=self.colors['input_bg'],
                                 fg=self.colors['text'],
                                 relief=tk.FLAT, wrap=tk.WORD)
        self.info_text.pack(fill=tk.X, padx=10, pady=5)

        # Canvas для визуализации
        canvas_frame = tk.Frame(right_panel, bg=self.colors['bg'])
        canvas_frame.pack(fill=tk.BOTH, expand=True)

        # Заголовок визуализации
        viz_label = tk.Label(canvas_frame, text="Визуализация кучи (Max-Heap)",
                            font=('Arial', 14, 'bold'),
                            bg=self.colors['bg'], fg=self.colors['text'])
        viz_label.pack(pady=10)

        # Canvas с поддержкой скроллинга
        self.canvas = tk.Canvas(canvas_frame, bg=self.colors['bg'],
                               highlightthickness=0)
        self.canvas.pack(fill=tk.BOTH, expand=True, padx=20, pady=10)

        # Состояние
        self.update_info()

    def on_module_change(self, event=None):
        """Смена модуля реализации"""
```

```
selected = self.module_var.get()

if selected == self.current_module:
    return

elements = []
if hasattr(self.heap, 'get_all_elements'):
    elements = self.heap.get_all_elements()

if selected == "Python":
    try:
        self.heap = PythonHeap()
        self.current_module = "Python"
    except Exception as e:
        messagebox.showerror("Ошибка", f"Не удалось загрузить Python модуль: {
        self.module_var.set(self.current_module)
        return
else:
    try:
        self.heap = CppHeapWrapper()
        self.current_module = "C++"

        # Проверяем, загрузился ли C++ модуль
        if hasattr(self.heap, '_python_heap') and self.heap._python_heap:
            messagebox.showwarning("Предупреждение",
                                   "C++ модуль не найден. Скомпилируйте heap_cpp
            self.heap = PythonHeap()
            self.current_module = "Python"
            self.module_var.set("Python")
            return

    except Exception as e:
        messagebox.showerror("Ошибка", f"Не удалось загрузить C++ модуль: {e}"
        self.module_var.set(self.current_module)
        return
```

```
        # Восстановление элементов
        for elem in elements:
            self.heap.push(elem)

        self.draw_heap()
        self.update_info()
        messagebox.showinfo("Успех", f"Модуль переключен на {selected}")

    def push_element(self):
        """Добавление элемента в кучу"""
        try:
            value = self.entry_value.get().strip()
            if not value:
                messagebox.showwarning("Предупреждение", "Введите значение")
                return

            value = int(value)
            self.heap.push(value)

            self.entry_value.delete(0, tk.END)
            self.draw_heap()
            self.update_info()
            self.show_message(f"Элемент {value} добавлен в кучу", "success")

        except ValueError:
            messagebox.showerror("Ошибка", "Введите корректное целое число")
        except Exception as e:
            messagebox.showerror("Ошибка", f"Не удалось добавить элемент: {e}")

    def pop_element(self):
        """Удаление корневого элемента"""
        try:
            if self.heap.is_empty():
                messagebox.showwarning("Предупреждение", "Куча пуста. Нечего удалять."
                return
```

```
        value = self.heap.pop()
        self.draw_heap()
        self.update_info()
        self.show_message(f"Удален корневой элемент: {value}", "success")

    except Exception as e:
        messagebox.showerror("Ошибка", f"Ошибка при удалении корня: {e}")

def remove_at_index(self):
    """Удаление элемента по индексу"""
    try:
        index = self.entry_index.get().strip()
        if not index:
            messagebox.showwarning("Предупреждение", "Введите индекс элемента")
            return

        index = int(index)

        if self.heap.is_empty():
            messagebox.showwarning("Предупреждение", "Куча пуста")
            return

        if index < 0 or index >= self.heap.size():
            messagebox.showerror("Ошибка",
                                 f"Неверный индекс. Допустимый диапазон: 0-{self.hea
            return

        if self.heap.remove_at(index):
            self.entry_index.delete(0, tk.END)
            self.draw_heap()
            self.update_info()
            self.show_message(f"Элемент с индексом {index} удален", "success")
        else:
            messagebox.showerror("Ошибка", "Не удалось удалить элемент")

    except ValueError:
```

```
        messagebox.showerror("Ошибка", "Введите корректный целочисленный индекс")
    except Exception as e:
        messagebox.showerror("Ошибка", f"Ошибка при удалении: {e}")

def clear_heap(self):
    """Очистка кучи"""
    if self.heap.size() > 0:
        # Python heap можно пересоздать
        if self.current_module == "Python":
            self.heap = PythonHeap()
        else:
            while not self.heap.is_empty():
                try:
                    self.heap.pop()
                except:
                    break

        self.draw_heap()
        self.update_info()
        self.show_message("Куча очищена", "success")

def draw_heap(self):
    """Отрисовка кучи"""
    self.canvas.delete("all")

    elements = self.heap.get_all_elements()
    if not elements:
        self.canvas.create_text(400, 200, text="Куча пуста",
                                font=('Arial', 16), fill='#888888')
        return

    canvas_width = self.canvas.winfo_width()
    canvas_height = self.canvas.winfo_height()

    if canvas_width <= 1:
        canvas_width = 800
```

```
if canvas_height <= 1:
    canvas_height = 500

# Расчет уровней дерева
num_elements = len(elements)
levels = math.ceil(math.log2(num_elements + 1))

# Параметры отрисовки
node_radius = 25
vertical_spacing = 80
horizontal_spacing = canvas_width // 4

# Отрисовка узлов и связей
positions = {}

for level in range(levels):
    nodes_in_level = min(2 ** level, num_elements - (2 ** level - 1))
    start_x = (canvas_width - (nodes_in_level - 1) * horizontal_spacing) // 2

    for pos in range(nodes_in_level):
        array_index = (2 ** level - 1) + pos
        if array_index >= num_elements:
            break

        x = start_x + pos * horizontal_spacing
        y = 50 + level * vertical_spacing

        # Рисуем связь с родителем
        if array_index > 0:
            parent_index = (array_index - 1) // 2
            if parent_index in positions:
                parent_x, parent_y = positions[parent_index]
                self.canvas.create_line(parent_x, parent_y + node_radius,
                                        x, y - node_radius,
                                        fill=self.colors['line'], width=2)
```

```
            # Рисуем узел
            value = elements[array_index]
            self.canvas.create_oval(x - node_radius, y - node_radius,
                                    x + node_radius, y + node_radius,
                                    fill=self.colors['node'],
                                    outline=self.colors['node_hover'],
                                    width=2)

            # Текст значения
            self.canvas.create_text(x, y, text=str(value),
                                    fill=self.colors['text'],
                                    font=('Arial', 12, 'bold'))

            # Индекс
            self.canvas.create_text(x, y + node_radius + 10,
                                    text=f"[{array_index}]",
                                    fill='#aaaaaa',
                                    font=('Arial', 8))

            positions[array_index] = (x, y)

def update_info(self):
    """Обновление информационной панели"""
    self.info_text.delete(1.0, tk.END)

    info = f"Модуль: {self.current_module}\n"
    info += f"Тип кучи: Max-Heap\n"
    info += f"Размер: {self.heap.size()}\n"
    info += f"Пустая: {'Да' if self.heap.is_empty() else 'Нет'}\n"

    if not self.heap.is_empty():
        elements = self.heap.get_all_elements()
        info += f"Корень: {elements[0] if elements else 'N/A'}\n"
        info += f"Элементы: {elements[:5]}{'...' if len(elements) > 5 else ''}\n"

    self.info_text.insert(1.0, info)
```

```
    def show_message(self, message, msg_type="info"):
        """Показать временное сообщение"""
        color = self.colors['success'] if msg_type == "success" else self.colors['erro

        # Создаем временную метку для сообщения
        msg_label = tk.Label(self.root, text=message,
                             bg=color, fg=self.colors['text'],
                             font=('Arial', 10), padx=20, pady=5)
        msg_label.place(relx=0.5, rely=0.95, anchor='s')

        # Удаляем через 2 секунды
        self.root.after(2000, msg_label.destroy)

def main():
    """Главная функция"""
    root = tk.Tk()
    app = HeapVisualizer(root)

    # Обработка закрытия окна
    def on_closing():
        if messagebox.askokcancel("Выход", "Вы действительно хотите выйти?"):
            root.destroy()

    root.protocol("WM_DELETE_WINDOW", on_closing)
    root.mainloop()

if __name__ == "__main__":
    main()
```

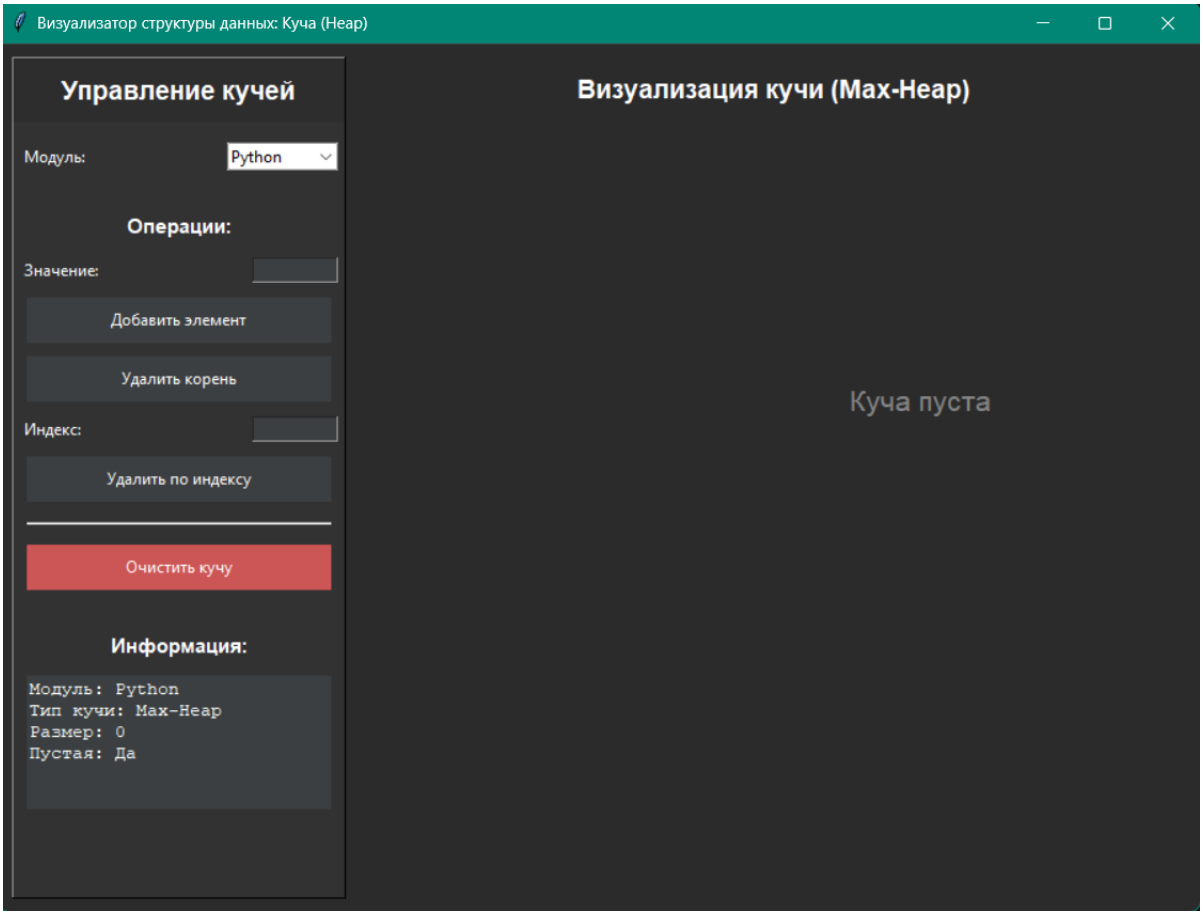

Рис. 1: Интерфейс

Исходный код модуля реализации кучи *heap_python_module.py*

```
import heapq
from typing import List

class PythonHeap:
    """Реализация кучи на Python с использованием heapq"""

    def __init__(self):
        self._heap: List[int] = []
        self._max_heap: List[int] = []  # Для имитации max-heap

    def push(self, value: int) -> None:
        """Добавление элемента в кучу"""
        heapq.heappush(self._max_heap, -value)
```

```
        self._heap.append(value)

    def pop(self) -> int:
        """Удаление и возврат корневого элемента"""
        if not self._max_heap:
            raise IndexError("Куча пуста")

        max_val = -heapq.heappop(self._max_heap)

        if max_val in self._heap:
            self._heap.remove(max_val)

        return max_val

    def top(self) -> int:
        """Получение корневого элемента без удаления"""
        if not self._max_heap:
            raise IndexError("Куча пуста")
        return -self._max_heap[0]

    def size(self) -> int:
        """Размер кучи"""
        return len(self._max_heap)

    def is_empty(self) -> bool:
        """Проверка на пустоту"""
        return len(self._max_heap) == 0

    def remove_at(self, index: int) -> bool:
        """Удаление элемента по индексу"""
        if index < 0 or index >= len(self._heap):
            return False

        value_to_remove = self._heap[index]
        self._heap.pop(index)
        self._max_heap.remove(-value_to_remove)
```

```
        heapq.heapify(self._max_heap)

        return True

    def get_all_elements(self) -> List[int]:
        """Получение всех элементов кучи"""
        return [-x for x in self._max_heap]

    def clear(self) -> None:
        """Очистка кучи"""
        self._heap.clear()
        self._max_heap.clear()
```

Исходный код реализации кучи heap_cpp_wrapper.py

```
# heap_cpp_wrapper.py
import ctypes
import os
import platform
from typing import List

class CppHeapWrapper:
    """Обертка для C++ модуля через ctypes"""

    def __init__(self):
        self._lib = None
        self._heap = None
        self._python_heap = False
        self._load_library()

    def _load_library(self):
        """Загрузка C++ библиотеки"""
        system = platform.system()

        if system == 'Windows':
            lib_name = 'heap_cpp_module.dll'
```

```
        elif system == 'Darwin':
            lib_name = 'libheap_cpp_module.dylib'
        else:
            lib_name = 'libheap_cpp_module.so'

        try:
            lib_path = os.path.join(os.path.dirname(__file__), lib_name)

            if not os.path.exists(lib_path):
                print(f"Библиотека {lib_name} не найдена. Будет использоваться Python
                self._python_heap = True
                return

            self._lib = ctypes.CDLL(lib_path)
            self._python_heap = False

            # Определение типов возвращаемых значений
            self._lib.create_heap.restype = ctypes.c_void_p
            self._lib.heap_size.restype = ctypes.c_int
            self._lib.heap_is_empty.restype = ctypes.c_bool
            self._lib.heap_top.restype = ctypes.c_int
            self._lib.heap_pop.restype = ctypes.c_int
            self._lib.heap_remove_at.restype = ctypes.c_bool
            self._lib.heap_get_all.restype = ctypes.POINTER(ctypes.c_int)

            # Создаем кучу
            self._heap = self._lib.create_heap()

        except Exception as e:
            print(f"Ошибка загрузки C++ библиотеки: {e}. Будет использоваться Python p
            self._python_heap = True
            self._lib = None

    def push(self, value: int) -> None:
        """Добавление элемента"""
        if self._lib and not self._python_heap:
```

```
        self._lib.heap_push(ctypes.c_void_p(self._heap), ctypes.c_int(value))
    else:
        raise RuntimeError("C++ модуль не загружен")

def pop(self) -> int:
    """Удаление корневого элемента"""
    if self._lib and not self._python_heap:
        if self.is_empty():
            raise IndexError("Куча пуста")
        return self._lib.heap_pop(ctypes.c_void_p(self._heap))
    else:
        raise RuntimeError("C++ модуль не загружен")

def top(self) -> int:
    """Получение корневого элемента"""
    if self._lib and not self._python_heap:
        if self.is_empty():
            raise IndexError("Куча пуста")
        return self._lib.heap_top(ctypes.c_void_p(self._heap))
    else:
        raise RuntimeError("C++ модуль не загружен")

def size(self) -> int:
    """Размер кучи"""
    if self._lib and not self._python_heap:
        return self._lib.heap_size(ctypes.c_void_p(self._heap))
    return 0

def is_empty(self) -> bool:
    """Проверка на пустоту"""
    if self._lib and not self._python_heap:
        return self._lib.heap_is_empty(ctypes.c_void_p(self._heap))
    return True

def remove_at(self, index: int) -> bool:
    """Удаление элемента по индексу"""
```

```
        if self._lib and not self._python_heap:
            return self._lib.heap_remove_at(ctypes.c_void_p(self._heap), ctypes.c_int(
        return False

    def get_all_elements(self) -> List[int]:
        """Получение всех элементов"""
        if self._lib and not self._python_heap:
            size = ctypes.c_int()
            arr_ptr = self._lib.heap_get_all(ctypes.c_void_p(self._heap), ctypes.byref

            if arr_ptr and size.value > 0:
                result = [arr_ptr[i] for i in range(size.value)]
                self._lib.free_array(arr_ptr)
                return result
        return []

    def __del__(self):
        """Деструктор для освобождения C++ памяти"""
        if self._lib and self._heap:
            self._lib.destroy_heap(ctypes.c_void_p(self._heap))
```

Исходный код реализации кучи *heap_cpp_module.cpp*

```
#include <vector>
#include <algorithm>
#include <stdexcept>

extern "C" {
    struct HeapStruct {
        std::vector<int>* data;
    };

    HeapStruct* create_heap() {
        HeapStruct* heap = new HeapStruct();
        heap->data = new std::vector<int>();
        return heap;
```

```
}

void destroy_heap(HeapStruct* heap) {
    delete heap->data;
    delete heap;
}

int heap_size(HeapStruct* heap) {
    return heap->data->size();
}

bool heap_is_empty(HeapStruct* heap) {
    return heap->data->empty();
}

int heap_top(HeapStruct* heap) {
    if (heap->data->empty()) {
        throw std::runtime_error("Куча пуста");
    }
    return (*heap->data)[0];
}

void heap_push(HeapStruct* heap, int value) {
    heap->data->push_back(value);
    std::push_heap(heap->data->begin(), heap->data->end());
}

int heap_pop(HeapStruct* heap) {
    if (heap->data->empty()) {
        throw std::runtime_error("Куча пуста");
    }
    std::pop_heap(heap->data->begin(), heap->data->end());
    int result = heap->data->back();
    heap->data->pop_back();
    return result;
}
```

```
    bool heap_remove_at(HeapStruct* heap, int index) {
        if (index < 0 || index >= heap->data->size()) {
            return false;
        }
        heap->data->erase(heap->data->begin() + index);
        std::make_heap(heap->data->begin(), heap->data->end());
        return true;
    }

    int* heap_get_all(HeapStruct* heap, int* size) {
        *size = heap->data->size();
        if (*size == 0) return nullptr;
        int* arr = new int[*size];
        std::copy(heap->data->begin(), heap->data->end(), arr);
        return arr;
    }

    void free_array(int* arr) {
        delete[] arr;
    }
}
```

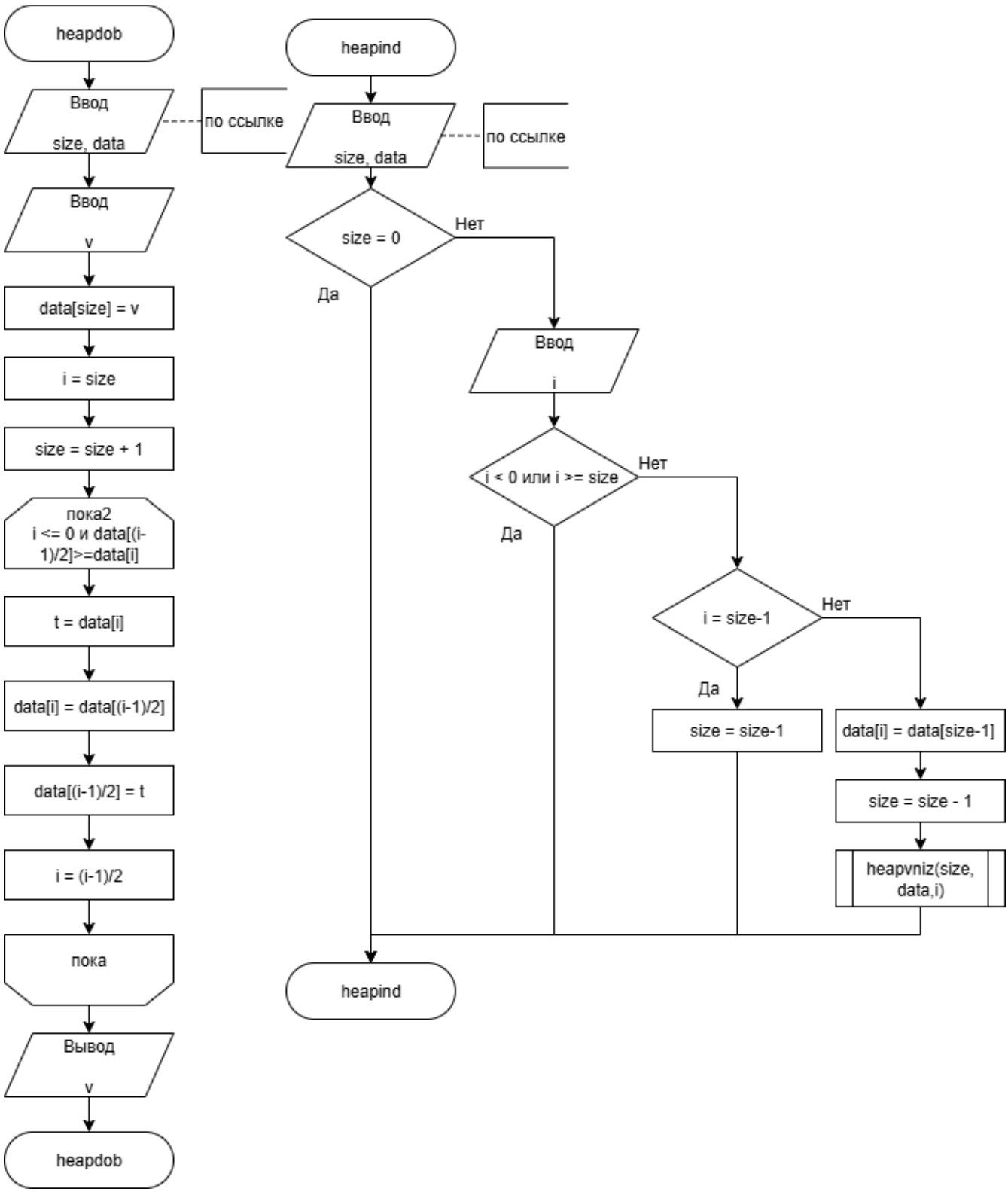

Рис. 2: Схема работы с кучей.

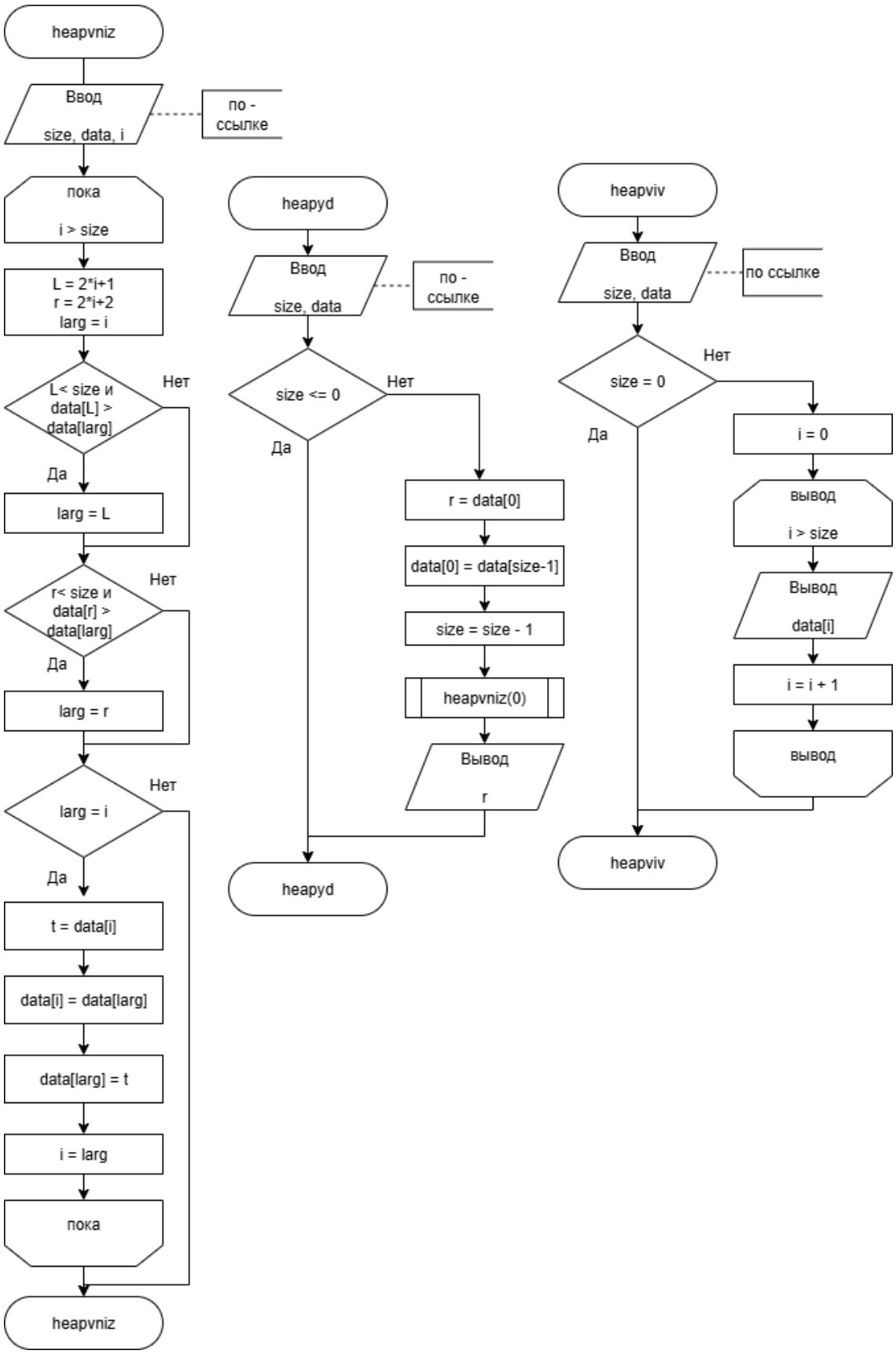

Рис. 3: Схема работы с кучей.

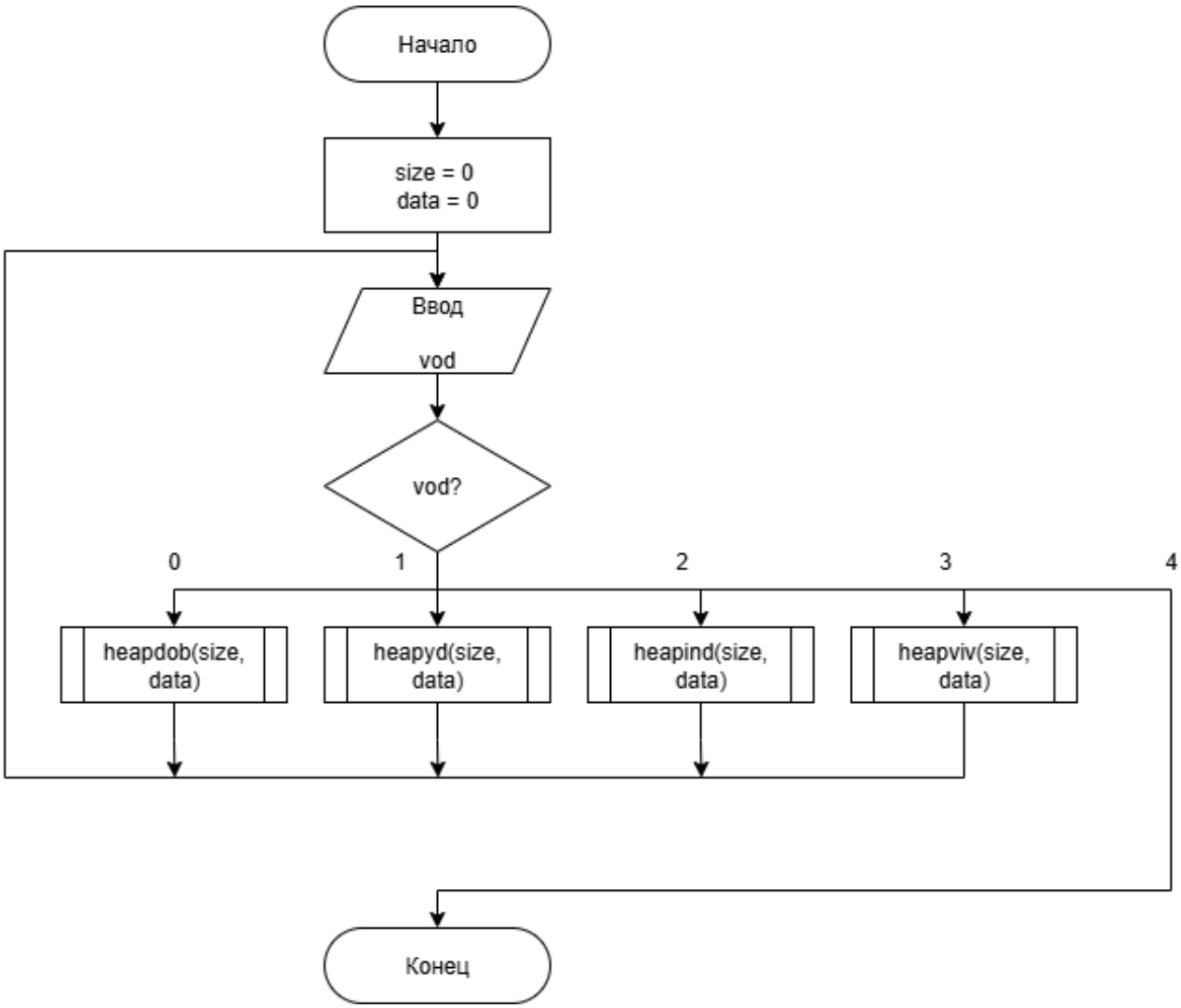

Рис. 4: Схема работы с кучей.

Особенности работы модулей

Обе библиотеки предоставляют функциональность двусторонней очереди с идентичным набором операций: вставка и извлечение элементов с обоих концов, определение размера, проверка пустоты, строковое представление и очистка. Каждый элемент хранит целочисленное значение и текстовую строку из 9 символов. В версиях на Python и C применён односвязный список с указателями head и tail, из-за чего операция pop_back требует O(n), поскольку отсутствует ссылка на предшествующий узел. В C-реализации управление памятью осуществляется через malloc/free.

Особенности работы модулей

Cpp-библиотека (ctypes). Библиотека собирается в динамический файл (dll для Windows) с экспортом функций через интерфейс C. В среде Python загружается посредством ctypes.Cdll(), после чего требуется вручную задать типы аргументов (argtypes) и возвращаемых значений (restype) для каждой функции. Взаимодействие построено на процедурном подходе: создание дека, выполнение операций и освобождение ресурсов вызываются как независимые функции.
Структура хранения: Дек представляет собой структуру с тремя полями — указатель head на первый узел, указатель tail на последний узел и счётчик size. Узлы формируют односвязный список, где каждый Node содержит целое число num, текстовый буфер text длиной 10 символов и указатель next на следующий узел. Конечный узел указывает на NULL.
Python-module. Никакой специальной интеграции не требуется, поскольку библиотека написана на чистом Python. Класс DequePython импортируется напрямую и эксплуатируется как стандартный Python-объект. Память управляется автоматически сборщиком мусора.
Структура хранения: Организация полностью повторяет C-версию. Класс DequePython хранит ссылки head и tail на объекты класса Node, а также

счётчик size. Каждый Node содержит поля number, text и ссылку next на следующий узел. Завершающий узел цепочки имеет next равный None.

Реализованные функции

Python: push, pop, top, size, is_empty, remove_at, get_all_elements, clear
C++: create_heap, destroy_heap, heap_push, heap_pop, heap_size, heap_is_empty, heap_top, heap_remove_at, heap_get_all, free_array

Вывод

На базе структуры данных, разработанной на языке C в рамках лабораторной работы 4.1, был выполнен перенос кода на C++. Подготовленный модуль был интегрирован в графический интерфейс, созданный с помощью библиотеки tkinter на языке Python, с использованием ctypes. Затем был реализован модуль на чистом Python и также подключён к GUI. Дополнительно был создан модуль на C++ с использованием готовых методов и контейнеров из STL, интегрированный через PyBind11. В результате были освоены новые компетенции: работа с PyBind, формирование статической библиотеки на C++, разработка графического интерфейса с помощью tkinter.